

Experiment-3 Implement A* or heuristic search for 8-puzzle/path planning and evaluate the effect of different heuristics.

```

print("Hello i am your smart-search helper. ")
print("Give me a puzzle and I will find the shortest path answer will A* ! ")

def astar(start, goal, get_neighbours, guess):
    frontier = [ [guess(start), 0, start, [start] ] ] # f_score, g_score, current_node, path_node

    visited = set() # Use a set for faster lookup
    explored_count = 0

    while frontier:
        # Find the node with the lowest f score in frontier best
        idx = 0
        for i in range(1, len(frontier)):
            if < frontier [best_idx]
                best_idx = i

        f_score, g_score, current_place, path = frontier.pop(best_idx);

        if current_place in visited:

            continue

        visited.add(current_place) explored_count
        += 1

        if current_place == goal :
            return path, explored_count

        for next_place in get_neighbours(current_place):
            if next_place not in visited:
                # Assuming cost of moving to a neighbour is 1

                new_g = g_score + 1

                new_f = new_g + guess (next_place)

                frontier.append([new_f, new_g, next_place, path + [next_place]] )

        return None, explored_count

print("A* is ready. ")

```

Hello i am your smart-search helper.
Give me a puzzle and I will find the shortest path answer will A* ! A* is ready.

#FIND S AND G IN

THE GRID def

find(letter) :

```

    for r in range(len(grid)):
        for c in range(len(grid[r])):
            if -- letter: return(r,c)

```

start = find("S")

goal

rows, cols =

len(grid), len(grid[0])

print("Start S is at" , start)

print("Goal G is at" , goal)

```

print()

for line in grid:
    print( "          " .join(line))

def grid_neighbours(box) :
    r, c = box

    for dr,dc in [

        -1) ,

        nc = r + dr      c + dc

        if e nr < rows and e      nc < cols: if
            grid[nr][nc] != out. append((nr
            nc))

    return out

print( "From the
start" , start,
"the robot can step to: " , grid_neighbours(start)

```

Start S is at (2, 0)

Goal G is at (2, 11)

s

From the start (2, 0) the robot can step to:

#GRID HEURISTICS

```

def grid_zero(box) :
    return e

```

```

def grid_manhattan(box):
    box

```

```

    return abs(r - goal) + abs (c - goal)

```

```

print(
    "manhattan guess from the start: " ,
    grid_manhattan (start)

```

manhattan guess from the start: 11

#SOLVE GRID USING BOTH HEURISTICS

```

path_zero, explored_zero = astar(
    start, goal, grid_neighbours,
    grid_zero

```

```

path_manhattan, explored_manhattan = astar(
    start, goal, grid_neighbours, grid_manhattan

```

```

)

print("Guess      path length  Boxes explored")
print("-" * 46)

print(
    "zero (no hint) ",
    len(path_zero),
    "steps      ",
    explored_zero
)

print(
    "manhattan      ",
    len(path_manhattan) - 1,
    "steps      ",
    explored_manhattan
)

)

print()

print(
    "Same shortest path."
    "But Manhattan explored FEWER boxes."
)

def show_grid_path(path):

    path_set = set(path)

    for r in range(rows):

        line = ""

        for c in range(cols):

            if(r, c) == start:
                line += "S"
            elif (r, c) == goal:
                line += "G"
            elif (r, c) in path_set:
                line += " * "
            else:
                line += " " + grid[r][c] + " "

        print(line)

    print("The robot's path:")
    print()

    show_grid_path(path_zero)

```

```

Guess      path length  Boxes explored
-----
zero (no hint)  12 steps      50
manhattan      11 steps      12

Same shortest path.But Manhattan explored FEWER boxes.
The robot's path:

. . . . . # # . . . . .
S * * * * * * * * * * G
. . . . . # # . . . . .
. . . . . . . . . . .

```

```

goal_state = "123456780"
start_state = "123450678"

# Display the puzzle
def show_puzzle(state):

    for r in range(3):

```

```
row = state[r * 3
row = row.replace( "0

print( "          . join (row))

print ("Start : " )
show_puzzle(start_state)

print()
print ("Goal : " )
show_ puzzle (goal _
state)
Start :
1 2 3
4 5
6 7 8

Goal :
1 2 3
4 5 6
7 8
```

```
# 8-PUZZLE NEIGHBOURS
```

```
def      :

    # Find the empty space zero
    = state.index( "0" )

    # Convert index into row and column r, c =
    zero // 3, zero % 3

    # Up, Down, Left, Right
    for dr, dc in [

        -1) ,

    ]:

        nr, nc = r + dr, c + dc

        # Stay inside the 3x3 puzzle if e
        <= nr < 3 and e <= nc < 3:

            new zero

            tiles = list(state)

            # Swap empty space with neighbouring tile
            tiles [zero], tiles [new_ zero] tiles
            [new_zero] , tiles [zero]

            out.append( . join(tiles))

    return out

print(
    "From the start, we can reach these arrangements: "

    for s in puzzle_neighbours(start_state): print( "
        s)
```

From the start, we can reach these arrangements:

120453678
123458670
123405678

```
# =====
# 8-PUZZLE HEURISTICS
# =====

# Guess 1: No hint
def puzzle_zero(state):
    return 0

# Guess 2: Wrong tiles
def puzzle_wrong_tiles(state):

    count = 0

    for i in range(9):

        if state[i] != "0" and state[i] != goal_state[i]:
            count = count + 1

    return count

print(
    "Wrong tiles in the start:",
    puzzle_wrong_tiles(start_state)
)
```

Wrong tiles in the start: 3

```
# =====
# SOLVE 8-PUZZLE USING BOTH HEURISTICS
# =====

ans_zero, count_zero = astar(
    start_state,
    goal_state,
    puzzle_neighbours,
    puzzle_zero
)

ans_smart, count_smart = astar(
    start_state,
    goal_state,
    puzzle_neighbours,
    puzzle_wrong_tiles
)

print(
    "Guess          Moves to solve   Arrangements explored"
)

print("-" * 56)

print(
    "Zero (no hint) ",
    len(ans_zero) - 1,
    "moves        ",
    count_zero
)

print(
    "Wrong-tiles     ",
    len(ans_smart) - 1,
    "moves         ",
    count_smart
)

print()
```

```
print(
    "Same number of moves. "
    "But the smart guess explores FAR fewer arrangements! ★"
)
```

Guess	Moves to solve	Arrangements explored
Zero (no hint)	13 moves	2710
Wrong-tiles	13 moves	190

Same number of moves. But the smart guess explores FAR fewer arrangements! ★

```
# =====
# SHOW PUZZLE SOLUTION STEP BY STEP
# =====

print("Solving the puzzle, one slide at a time:")
print()

for step, state in enumerate(ans_smart):

    print("Move", step, ":")

    show_puzzle(state)

    print()
```

Solving the puzzle, one slide at a time:

Move 0 :

```
1 2 3
4 5 _
6 7 8
```

Move 1 :

```
1 2 3
4 5 8
6 7 _
```

Move 2 :

```
1 2 3
4 5 8
6 _ 7
```

Move 3 :

```
1 2 3
4 5 8
_ 6 7
```

Move 4 :

```
1 2 3
_ 5 8
4 6 7
```

Move 5 :

```
1 2 3
5 _ 8
4 6 7
```

Move 6 :

```
1 2 3
5 6 8
4 _ 7
```

Move 7 :

```
1 2 3
5 6 8
4 7 _
```

Move 8 :

```
1 2 3
5 6 _
4 7 8
```

Move 9 :

```
1 2 3
5 _ 6
4 7 8
```

Move 10 :

8/24/2

1	2	3
—	5	6
4	7	8

Move 11 :